Online Retail Sales Trend Analysis Using Python

Objective

The aim of this project was to analyze transaction-level retail data and identify when sales were highest over time. The analysis focused on understanding monthly, daily, and weekday revenue patterns using Python, with the goal of producing business insights that could support stock planning, staffing, and sales forecasting.

Data Preparation

The analysis used the UCI Online Retail dataset, which contains 541,909 transactions from a UK-based non-store online retailer between 01/12/2010 and 09/12/2011. The data is well suited to time-based sales analysis because it records both the date and time of each transaction, along with quantity and unit price information. The UCI documentation also notes that invoice numbers beginning with "C" represent cancellations, which is important for data cleaning.

To prepare the data for analysis, I removed rows with missing Customer ID, filtered out cancelled transactions, and excluded transactions with non-positive quantity or unit price. After cleaning, the dataset contained 397,884 rows and 9 columns, including the newly created revenue variable Total Price.

Monthly Sales Trend

Monthly revenue was calculated by summing Total Price across all transactions for each month. The results show that sales were relatively moderate in the earlier part of the period but increased strongly in the second half of 2011.

```
monthly_sales = df.groupby("InvoiceMonth")["TotalPrice"].sum().reset_index()
monthly_sales["InvoiceMonth"] = monthly_sales["InvoiceMonth"].astype(str)

print(monthly_sales)
```

```
   InvoiceMonth   TotalPrice
0       2010-12   572713.890
1       2011-01   569445.040
2       2011-02   447137.350
3       2011-03   595500.760
4       2011-04   469200.361
5       2011-05   678594.560
6       2011-06   661213.690
7       2011-07   600091.011
8       2011-08   645343.900
9       2011-09   952838.382
10      2011-10  1039318.790
11      2011-11  1161817.380
12      2011-12   518192.790
```

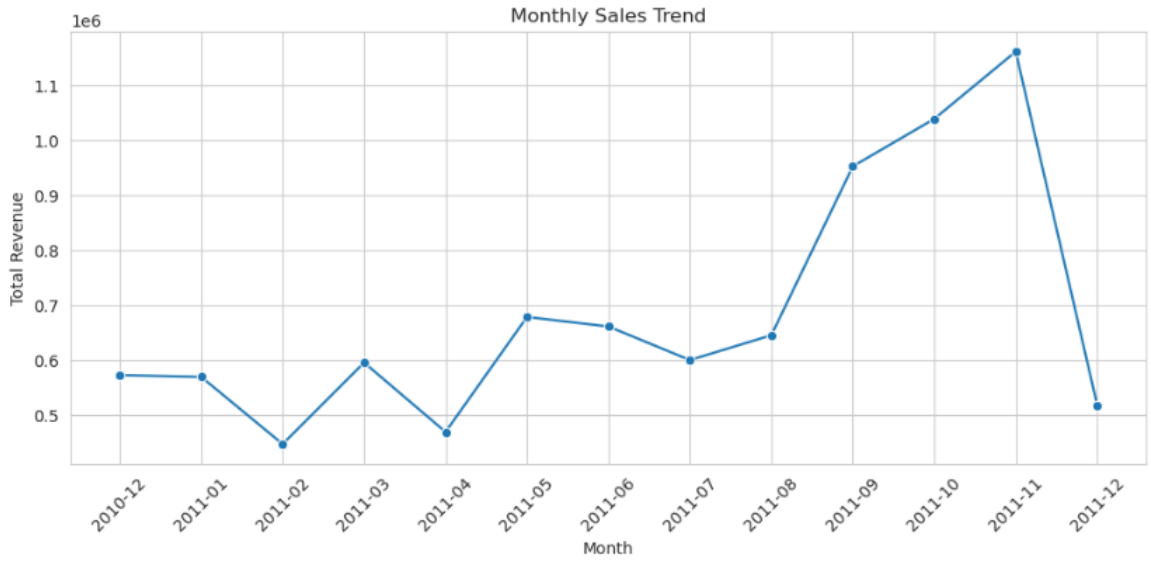
The lowest monthly revenue in the printed table occurred in February 2011 at approximately 447,137.35, while the highest monthly revenue occurred in November 2011 at approximately 1,161,817.38

```
monthly_sales = df.groupby("InvoiceMonth")["TotalPrice"].sum().reset_index()
monthly_sales["InvoiceMonth"] = monthly_sales["InvoiceMonth"].astype(str)

print(monthly_sales)
```

```
   InvoiceMonth   TotalPrice
0       2010-12   572713.890
1       2011-01   569445.040
2       2011-02   447137.350
3       2011-03   595500.760
4       2011-04   469200.361
5       2011-05   678594.560
6       2011-06   661213.690
7       2011-07   600091.011
8       2011-08   645343.900
9       2011-09   952838.382
10      2011-10  1039318.790
11      2011-11  1161817.380
12      2011-12   518192.790
```

```
plt.figure(figsize=(10,5))
sns.lineplot(data=monthly_sales, x="InvoiceMonth", y="TotalPrice", marker="o")
plt.title("Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Total Revenue")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



The line chart supports this pattern clearly. Revenue rises steadily from late summer to autumn, peaks in November, and then falls in December. However, this drop in December should be interpreted carefully because the dataset ends on 09/12/2011, so the final month is incomplete rather than a full trading month.

Daily Sales Pattern

To explore shorter-term fluctuations, I aggregated total revenue by each transaction date. The daily sales chart shows substantial day-to-day variation throughout the period, with sales generally becoming stronger in the later months of the year. The highest single day of revenue was 09/12/2011, with total sales of approximately 184,349.28. This is consistent with the strong

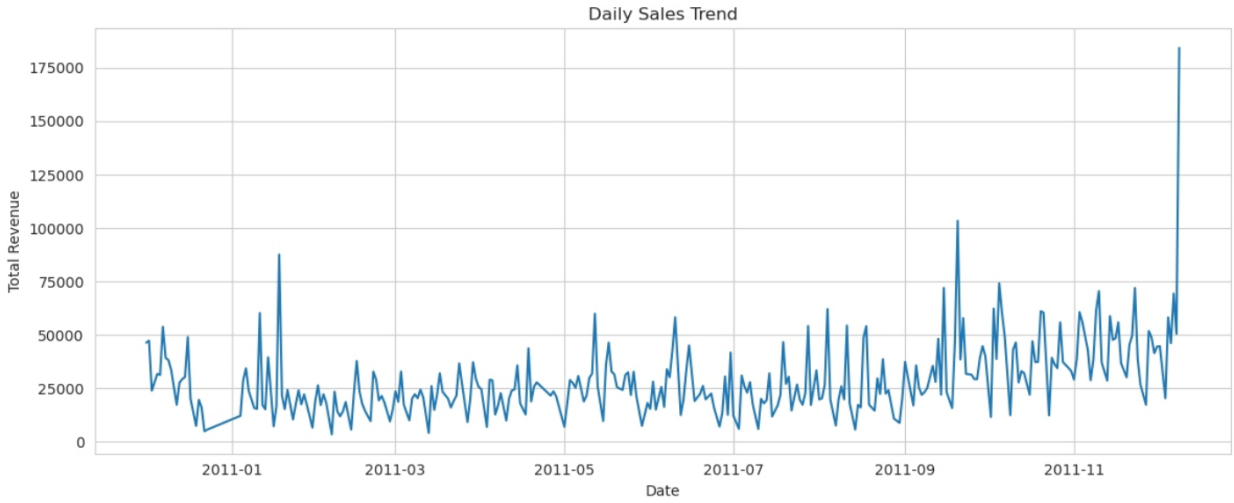
late-year pattern seen in the monthly trend, and it suggests a final surge in activity near the end of the available data window.

```
daily_sales = df.groupby("InvoiceDay")["TotalPrice"].sum().reset_index()

print(daily_sales.head())
```

```
   InvoiceDay  TotalPrice
0  2010-12-01    46376.49
1  2010-12-02    47316.53
2  2010-12-03    23921.71
3  2010-12-05    31771.60
4  2010-12-06    31215.64
```

```
plt.figure(figsize=(12,5))
sns.lineplot(data=daily_sales, x="InvoiceDay", y="TotalPrice")
plt.title("Daily Sales Trend")
plt.xlabel("Date")
plt.ylabel("Total Revenue")
plt.tight_layout()
plt.show()
```



Weekday Sales Pattern

Sales were also aggregated by day of the week to identify recurring weekly patterns. The weekday results show that Thursday generated the highest revenue, at approximately 1,976,859.07. Tuesday and Wednesday also performed strongly, while Sunday generated the lowest revenue among the recorded days. Saturday appears as NaN in the table after cleaning, which indicates that no valid Saturday transactions remained in the filtered dataset. This weekday pattern suggests that business activity was strongest in the middle of the week, which may be useful for staffing and promotional planning.

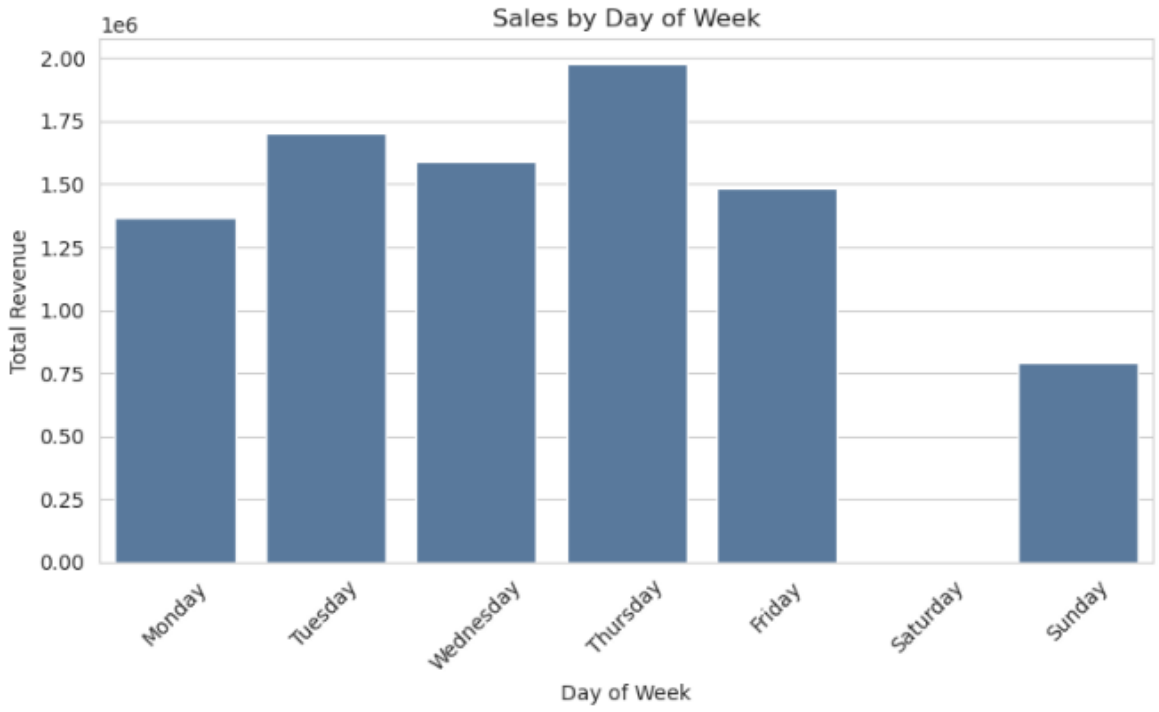
```
day_order = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"]

sales_by_day = df.groupby("DayName")["TotalPrice"].sum().reindex(day_order).reset_index()

print(sales_by_day)
```

```
     DayName   TotalPrice
0     Monday  1367146.411
1    Tuesday  1700634.631
2  Wednesday  1588336.170
3   Thursday  1976859.070
4     Friday  1485917.401
5   Saturday          NaN
6     Sunday   792514.221
```

```
plt.figure(figsize=(8,5))
sns.barplot(data=sales_by_day, x="DayName", y="TotalPrice", color="#4E79A7")
plt.title("Sales by Day of Week")
plt.xlabel("Day of Week")
plt.ylabel("Total Revenue")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



Key Findings

The analysis identified three main findings.

- First, November 2011 was the peak month for sales, with revenue of about 1.16 million.
- Second, 09/12/2011 was the highest single day in the dataset, with revenue of about 184,349.28.
- Third, Thursday was the strongest weekday overall, generating about 1.98 million in revenue.
- Overall, the results show that sales rose sharply in the final months of the year, indicating a strong seasonal uplift.
- This trend may reflect year-end purchasing behaviour, stronger trading conditions, or higher order volumes from wholesale customers, which is consistent with the dataset description that many customers are wholesalers.

Business Implications

- The findings have several practical implications.
- Since sales are strongest in the final months of the year, the business could plan inventory, staffing, and promotional activity more aggressively for the September to November period.
- The weekday analysis suggests that Thursday is a particularly important trading day, so operational resources could be prioritized midweek to handle higher demand.
- The spike at the end of the dataset also shows that late-period activity can rise quickly, so the business would benefit from monitoring near-real-time sales patterns during peak trading periods.

Conclusion

This analysis is based on a historical dataset, so the final month is incomplete and hence, it should not be compared directly with the previous months. In addition, the analysis focuses on revenue only, so it does not examine profit margins, product categories, or customer segments. These would be valuable extensions for future work.

From a business perspective, these results suggest that the retailer should prepare for stronger demand in late-year trading periods and pay particular attention to midweek sales activity. The project therefore provides a simple but useful example of how data analysis can support operational planning and business decision-making.